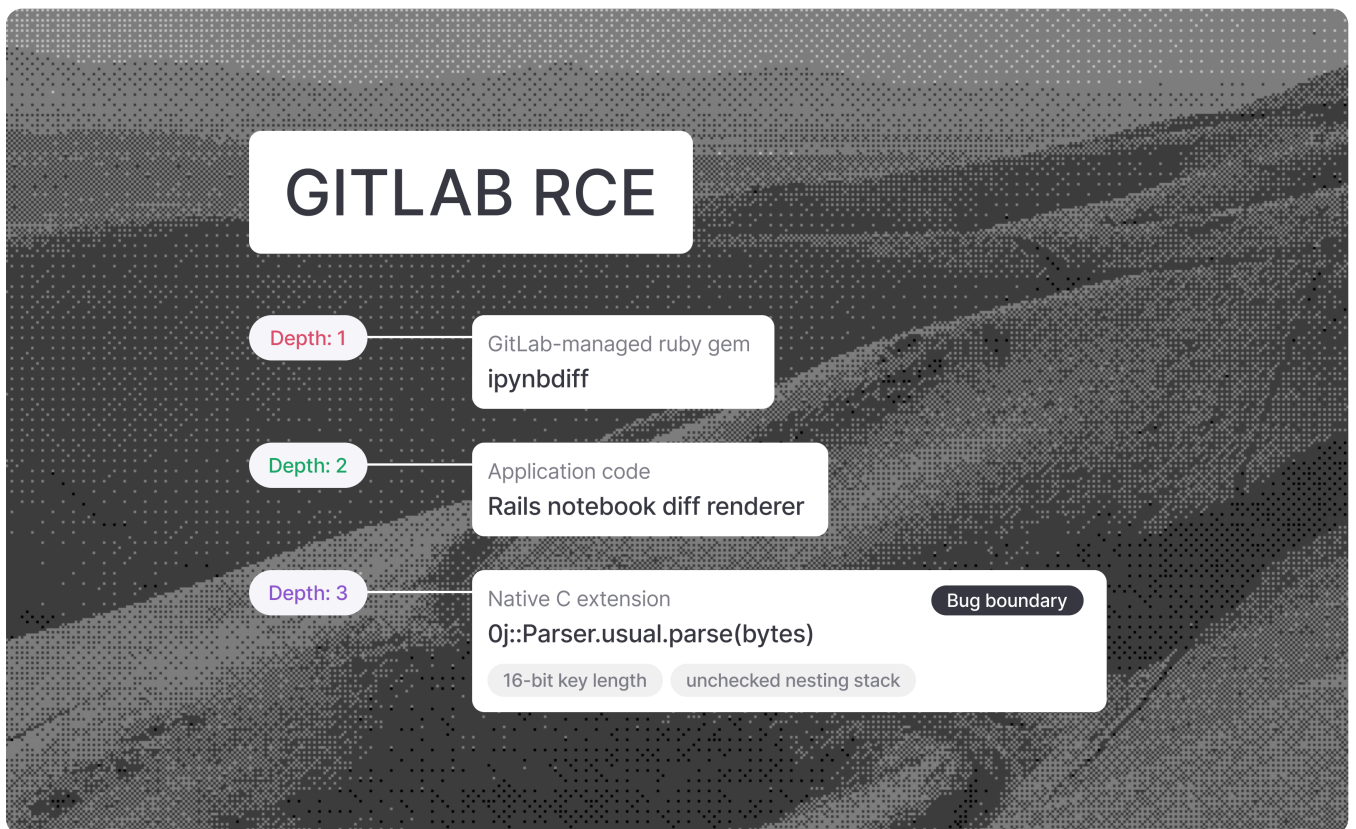


Погружение в глубину: как получить RCE в GitLab с помощью двух уязвимостей, связанных с повреждением памяти в Ruby



Опубликованный

24 июля 2026 года



Исследователь в области безопасности

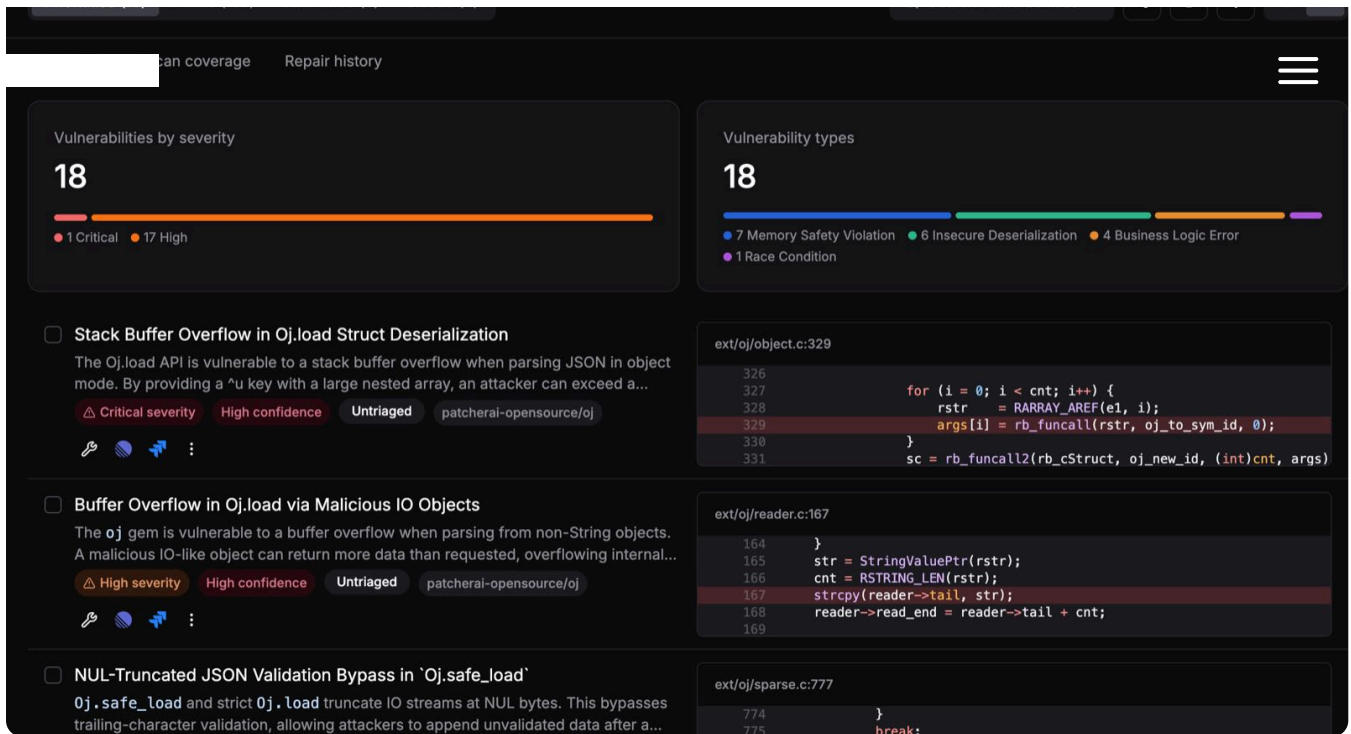
TL;DR: В рамках инициативы Open Defense Initiative система depthfirst проанализировала Oj, высокопроизводительный парсер JSON с обширной реализацией на языке C, и сформировала приоритетную очередь потенциально уязвимых находок. Система выявила **18** уязвимостей с высоким приоритетом, в том числе **7** ошибок, связанных с безопасностью памяти. Две из них — выход за границы массива и раскрытие указателя кучи — оставались в Oj почти пять лет. Oj — это низкоуровневая зависимость, используемая в GitLab, и мы объединили две ошибки, чтобы добиться удаленного выполнения кода в стандартной установке GitLab. В результате уязвимость затронула версии GitLab CE и EE с 15.2.0 по 18.10.7, с 18.11.0 по 18.11.4 и с 19.0.0 по 19.0.1. GitLab выпустила исправления вскоре после получения наших отчетов. Демонстрационный код доступен [здесь](#). Именно для решения таких задач и предназначен метод depthfirst: он позволяет глубоко исследовать реальные приложения, выявляя критические ошибки с высокой вероятностью возникновения и связывая их с достижимым влиянием на продукт.

Когда я думаю об уязвимостях, связанных с удаленным выполнением кода в GitLab, первое, что приходит мне на ум, — это старая схема «веб-приложение — Redis»: превратить веб-примитив в SSRF, использовать SSRF для работы с протоколом Redis, а затем убедить доверенный процесс выполнить работу из очереди. В отчете 2018 года о зеркалировании репозитория GitLab этот путь был продемонстрирован с помощью внедрения CRLF в специально сформированный `git://` URL в самостоятельно управляемых системах, где Redis был доступен по протоколу TCP. Сегодня GitLab защищен несколькими уровнями защиты от SSRF. Если старый маршрут перекрыт, по какому еще пути можно прорваться?

GitLab написан на Ruby, который обычно считается безопасным с точки зрения работы с памятью. Автоматическое управление памятью и проверка

первую очередь на логике приложения, а не на ручном выделении памяти, жизни объектов или границах массивов. Просматривая результаты сканирования в рамках нашей [инициативы Open Defense](#), мы заметили, что во многих проектах Ruby gem есть `ext/` каталог, полный кода на C, с большим количеством обнаружений повреждения памяти. То, что выглядит как обычная обработка Ruby, – это перемещение объектов `String`, `Array` и `Hash` в собственные анализаторы, полные указателей, буферов фиксированного размера, преобразований целых чисел и времени жизни, управляемого вручную. Это открытие удивило нас и открыло новое направление исследований в области безопасности: может ли злоумышленник использовать уязвимый gem для запуска повреждения памяти в приложениях Ruby?

Мы проверили, какие гемы использует GitLab, и среди них оказался Oj. Oj — это высокопроизводительный парсер JSON для Ruby, реализованный в основном как нативное расширение на языке C и используемый GitLab и многими другими Ruby-приложениями. Мы просканировали исходный код Oj с помощью системы поиска в глубину. Система проанализировала нативную реализацию в целом и свела ее к приоритетной очереди, в которой для каждой уязвимости сохранялись исходное местоположение, окружающий код и вероятный класс ошибки. В итоге в приоритетную очередь попали 18 уязвимостей, в том числе 7 нарушений безопасности памяти. Таким образом, обзор был сосредоточен на буферах фиксированного размера, изменениях целочисленной ширины, времени жизни объектов и состоянии парсера, общем для всех обратных вызовов.



Система depthfirst выявила 18 приоритетных уязвимостей в Oj, в том числе семь ошибок, связанных с безопасностью памяти.

При сравнении этой очереди с исходным кодом Oj и поведением во время выполнения было выявлено несколько конкретных ошибок, связанных с безопасностью памяти. Две из них образовали цепочку, описанную в этом посте: непроверяемый стек вложенности на `Oj::Parser.usual.parse` предоставлял примитив записи, а непроверенное сужение ключа по длине приводило к раскрытию указателя на кучу.

Tracing Oj Back to GitLab

Starting from the affected Ruby entry point, `Oj::Parser.usual.parse`, we searched GitLab's source tree for production call sites. One immediately stood out: a small GitLab-managed, in-tree gem called `ipynbdiff`.

The name is descriptive: `.ipynb` is the file extension for Jupyter Notebooks, and `diff` refers to comparing two revisions of a file. To understand why this code invokes a JSON parser, it helps to first look at what GitLab sees when someone commits a notebook.

What Lives Inside an `.ipynb` File

scientific computing, and machine-learning work. To its author, it looks like a .ipynb file is a collection of cells. One cell may contain explanatory Markdown, another may contain Python or another language, and a code cell may retain the text, tables, plots, or errors produced when it ran.

The file stored in Git is a JSON document. Its four important top-level fields are `cells`, `metadata`, `nbformat`, and `nbformat_minor`. Every cell has a `type`, `metadata`, and `source text`; code cells can additionally carry an `execution count` and `saved outputs`. A minimal notebook looks like this:

```
{
  "cells": [
    {
      "cell_type": "code",
      "execution_count": null,
      "id": "hello",
      "metadata": {},
      "outputs": [],
      "source": ["print(\"hello\")\n"]
    }
  ],
  "metadata": {},
  "nbformat": 4,
  "nbformat_minor": 5
}
```

The [official notebook format](#) allows much richer cells and output objects, including images and other MIME-typed data. Cell execution happens in Jupyter; GitLab's diff renderer works from the JSON already stored in the repository.

That JSON is convenient for software and awkward for code review. A one-line change in a code cell can sit among escaped newlines, execution counters, metadata, and large saved outputs. GitLab therefore offers a [cleaner notebook diff](#): it transforms the machine-readable `.ipynb` file into a human-readable representation organized around cells, source, and output.

walks the `cells` array, converts notebook metadata, cell headers, source text, puts into ordered text blocks, and then compares the transformed versions. Parsing the repository file is the first step, which is how the Oj call entered this feature.

Following One Commit Diff into Puma

Consider an authenticated project member who can push commits and view them. They commit one version of a notebook, modify it in the next commit, and open the second commit's diff page. The browser requests `GET /<project>/-/commit/<sha>/diffs_stream?offset=0` to load the rendered changes.

In a standard GitLab request path, [Workhorse](#) receives the HTTP request and proxies it to the Rails application. Rails runs inside [Puma](#), whose long-lived Ruby worker processes also load native extensions such as Oj. GitLab's repository layer, backed by [Gitaly](#), supplies the Git objects needed for the diff.

A Git blob is the byte content of a file at one revision. For a modified file, the old blob comes from the parent side of the diff and the new blob comes from the commit being viewed. When the file qualifies for notebook rendering, `Gitlab::Diff::Rendered::Notebook::DiffFile#notebook_diff` passes those contents to the in-tree gem ([pinned GitLab source](#)):

```
IpynbDiff.diff(
  source_diff.old_blob&.data,
  source_diff.new_blob&.data,
  ...
)
```

In Ruby, `&.` is the safe-navigation operator. It calls `.data` when the blob exists and yields `nil` when that side is absent. An added notebook therefore has only a new side; a deleted notebook has only an old side. `Transformer#transform` turns the missing side into an empty transformed notebook. A modified notebook supplies both byte strings.

in one commit and the child version in the next. `ipynbdiff.diff` transforms `fore` to `to`, giving a stable old-then-new order. At this point, both `non-nil` arguments are still raw repository bytes.

The Ruby Line That Entered C

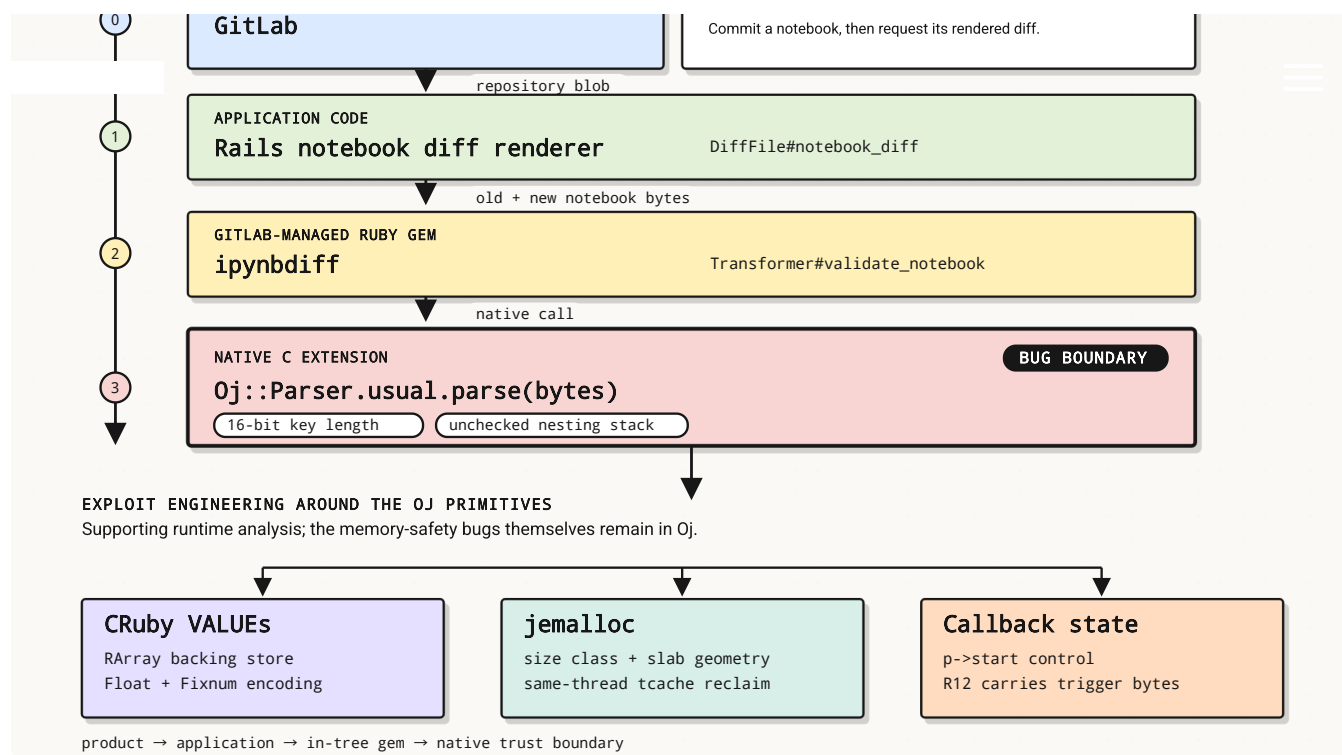
Inside `ipynbdiff`, every existing notebook side passes through `Transformer#transform`. Before the gem can walk its cells, it calls `validate_notebook`. The method performs a minimal parse-and-shape check ([pinned transformer source](#)):

```
def validate_notebook(notebook)
  notebook_json = Oj::Parser.usual.parse(notebook)
  return notebook_json if notebook_json&.key?('cells')
  raise InvalidNotebookError
rescue EncodingError, Oj::ParseError, JSON::ParserError
  raise InvalidNotebookError
end
```

The `notebook` argument is a Ruby `String` backed by the repository blob bytes. `Oj::Parser.usual` returns a process-wide native parser singleton. Each Puma worker process has its own copy, while threads in that worker share it. Calling `.parse` passes a pointer to the `String`'s byte buffer into `Oj`'s C extension, which constructs Ruby objects from the JSON. If parsing succeeds, `validate_notebook` then checks whether the top-level object contains a `cells` key.

That completed the reachability proof. A normal authenticated user able to create or push to a project and view the resulting commit diff could commit an `.ipynb` file and trigger GitLab's notebook renderer by opening that diff, causing repository-controlled bytes to reach `Oj` inside the Puma worker.

The investigation ran in reverse: `depthfirst` found the vulnerable parser below the application, then traced its callers back to a user-controlled product boundary. A real request follows that chain in the opposite direction, from a GitLab commit diff into native C code.

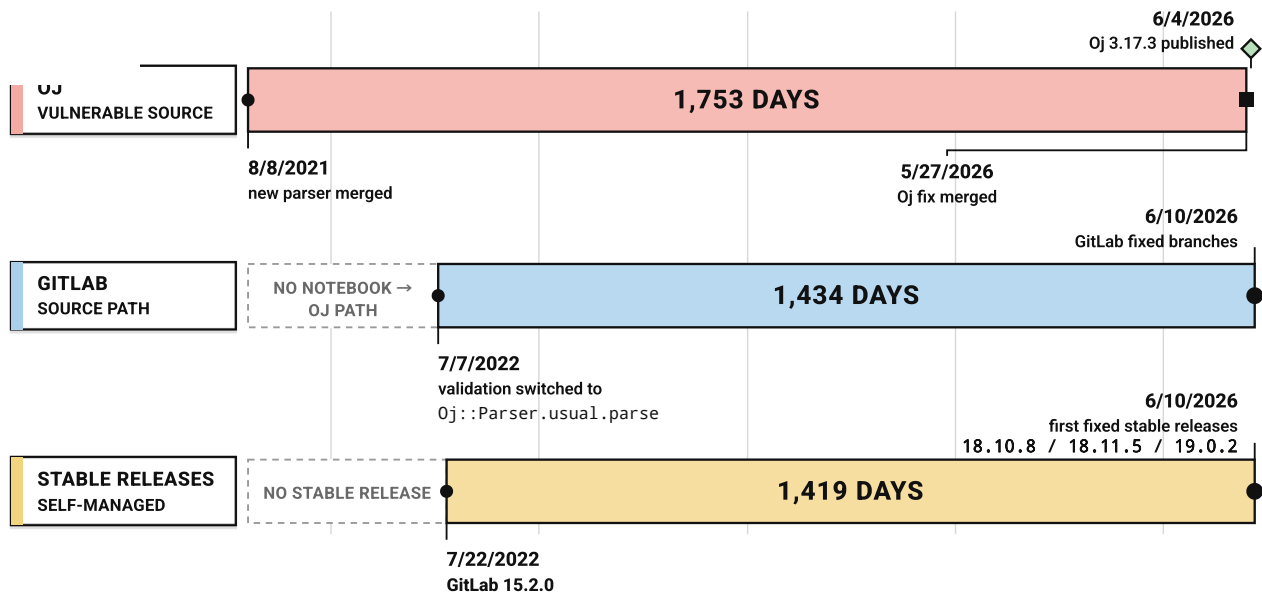


Notebook bytes cross GitLab's product boundary, Rails renderer, GitLab-managed ipynbdiff gem, and Oj's native parser; CRuby, jemalloc, and callback state belong to exploit engineering around the Oj bugs.

Nearly 4 Years in the Notebook Renderer

The vulnerable parser code entered Oj on August 8, 2021, and first shipped in Oj 3.13.0. GitLab switched notebook validation to `Oj::Parser.usual.parse` in July 2022, making the vulnerable path reachable in self-managed GitLab beginning with version 15.2.0.

The bugs remained in Oj for 1,753 days before the upstream fixes were merged, following depthfirst's report through the Open Defense Initiative. The vulnerable notebook path shipped in GitLab releases for nearly four years. GitLab triaged the report quickly and fixed its maintained release lines by upgrading to Oj 3.17.3. The fixes were merged upstream on May 27, published in Oj 3.17.3 on June 4, and shipped in GitLab's June 10 security releases.



Calendar-aligned source and release windows distinguish the Oj fix merge, the first published fixed gem, and GitLab's fixed branches.

I. Write: From Repeated `0x01` Bytes to a Chosen `p→start`

The nesting bug began with a narrow primitive. Once `depth` passed the end of the nesting stack, Oj continued writing one-byte collection selectors into the fields that followed it. The demonstrated payload used only arrays, so nesting depth controlled where a contiguous sweep of `0x01` bytes stopped. It could not place an arbitrary pointer directly into the parser.

The exploitation problem was therefore to find parser state that could make this restricted write useful. `buf.head`, the parser's buffer-start pointer, provided that bridge. The sections below follow the transformation in order: the selector overflow, the pointer corruption and allocator handoff, and finally the Ruby Array overlap that reached `p→start`.

The Primitive: A Forward Sweep of `0x01` Bytes

Oj 3.16.16 kept collection nesting state in a fixed 1,024-byte array inside the parser object:

```

    const char *map;
    const char *next_map;
    int depth;
    unsigned char stack[1024];

    struct _num num;
    struct _buf key;
    struct _buf buf;
    struct _funcs funcs[3];
    void (*start)(struct _ojParser *p);
    /* ... */
} *ojParser;

```

Opening an array selected the current callback table, incremented `depth`, and recorded the new collection type:

```

p->funcs[p->stack[p->depth]].open_array(p);
p->depth++;
p->stack[p->depth] = ARRAY_FUN;

```

Oj defined the three selectors next to the parser structure ([pinned source](#)):

```

#define TOP_FUN      0
#define ARRAY_FUN    1
#define OBJECT_FUN    2

```

The valid nesting indices were `stack[0]` through `stack[1023]`. Oj performed no depth check before assigning the selector for a newly opened collection. An array stored `0x01`; an object stored `0x02`. The demonstrated payload used arrays, so every level wrote `0x01` at: `P + 0x14 + depth`. The traversal sustained itself. On the next [, the byte just written was read back as the valid Array callback selector, allowing the parser to advance another level.

The demonstrated Writer 1 was a forward sweep of `0x01` bytes. Nesting depth chose where that sweep stopped, while every byte between the end of `stack`

or place a gadget address directly into `p→start`. The useful move was to write the legal `0x01` on a pointer that would determine where Writer 2 landed.

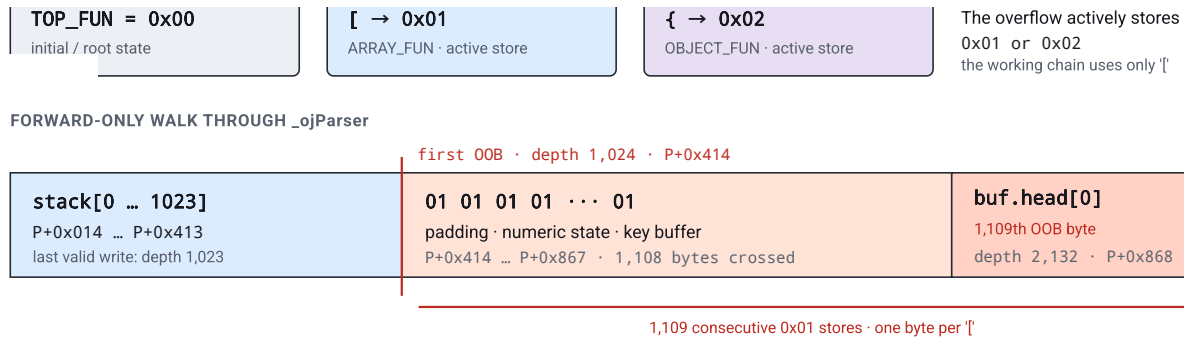
The Pivot: One Allowed Byte Repoints `buf.head`

The target was not `p→start`; Writer 1 could not encode that address. The target was `buf.head`, because later code would trust it as an allocation pointer.

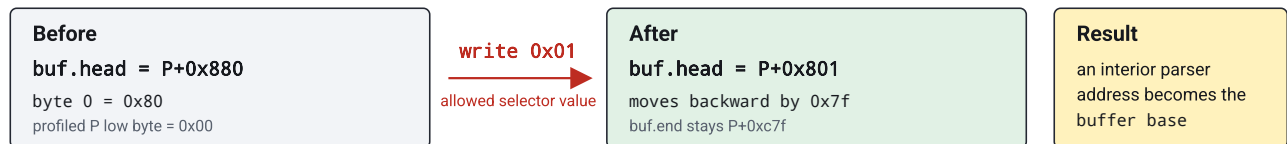
Depth 1,024 produced the first out-of-bounds write at `P+0x414`. From there, every `[` advanced the address by one and filled the entire range with `0x01`. The sweep crossed padding, numeric state, and the key buffer. The all-array prefix kept using Oj's separately allocated collection and VALUE stacks; it parsed no object key, and the following number reinitialized the numeric fields it needed.

The useful target was byte 0 of `buf.head` at `P+0x868`. It was reached at depth 2,132 ($0x868 - 0x14 = 0x854 = 2132$), making it the 1,109th byte written beyond `stack`. Before corruption, `buf.head` and `buf.tail` both pointed to the inline buffer at `P+0x880`, while `buf.end` was `P+0xc7f`.

In the profiled 3,584-byte jemalloc class, the parser region started with a zero low byte. The allowed value `0x01` therefore changed the head pointer's low byte from `0x80` to `0x01` ($P+0x880 \rightarrow P+0x801$), while `buf.end` and `buf.tail` remained unchanged. One selector byte had moved the logical buffer start backward by `0x7f`, exposing an interior address as Oj's buffer base. That pointer field was the amplifier: subsequent buffer code would pass the forged address to the allocator.



THE USEFUL BYTE



Moving buf.head backward by 0x7f turns the selector sweep into an allocator primitive.

A run of 2,132 array selectors advances 1,109 bytes beyond stack and changes buf.head from P+0x880 to P+0x801.

The Allocator Handoff: realloc Caches P+0x801

The allocator handoff had one goal: make the interior address something a normal allocation could later receive. The 4,001-digit number was chosen to force Oj to call `realloc` on the forged `buf.head`.

Three allocator sizes matter here. `_ojParser` itself occupied `0xdf0`, or 3,568 bytes. Jemalloc rounded that request to one `0xe00` 3,584-byte region. Eight such regions filled a `0x7000` seven-page slab. Releasing one region returned it to jemalloc's cache; it did not unmap the surrounding slab.

The forged head was still inside the parser region. With `buf.head = P+0x801` and `buf.end = P+0xc7f`, Oj calculated a capacity of `0x47e`, or 1,150 bytes. The payload next supplied the 4,001-digit number 1 followed by 4,000 2 digits.

Oj retained the first 18 digits in `_num`. When the number switched to the large-number path, `big_change()` set `buf.tail = buf.head` and reconstructed those 18 digits at the forged address. The parser scanned the remaining 3,983 digits as one span and passed them to a single `buf_append_string()` call. That call checked capacity before its bulk copy, so it entered the growth path with:

$$1150 + 3983 + 1150/2 = 5708 \text{ bytes}$$

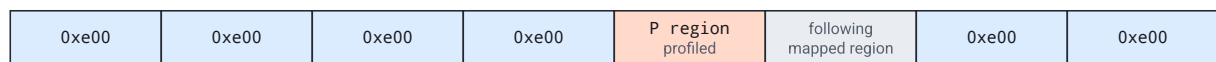
`realloc(P+0x801, 5708)` . In the profiled GitLab jemalloc build, this moved the buffer into the 6,144-byte class and cached the exact old address, `P+0x801` , at the top of the thread-local 3,584-byte bin.

Jemalloc treated the forged pointer as a 3,584-byte region and copied `[P+0x801, P+0x1601)` to the new allocation. The source extended `0x801` bytes beyond the parser's original region into the following slab region. Heap profiling established that this full range was mapped in the target placement. This interior-pointer behavior was specific to the measured allocator and build; it is not a portable `realloc` guarantee.

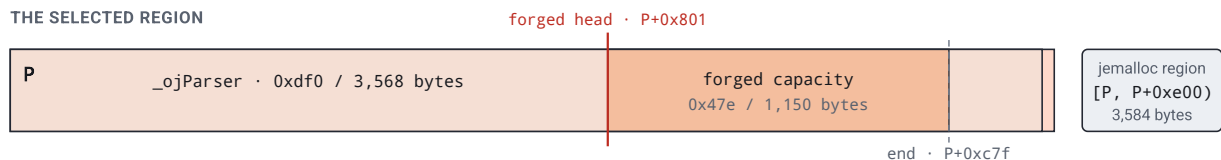
The following inner `[` advanced depth once more and wrote `0x01` at `P+0x869` , byte 1 of the now-moved `buf.head` pointer. The remaining sequence did not dereference that pointer again before arming the callback.

ONE 3,584-BYTE REGION INSIDE A SEVEN-PAGE SLAB

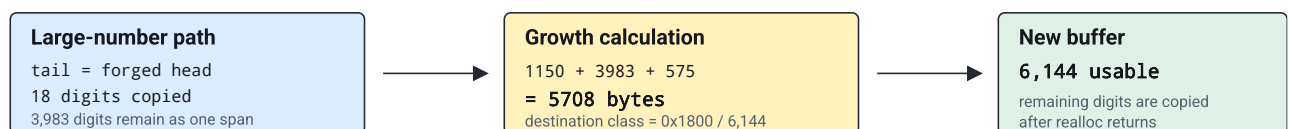
8 × 0xe00 = 0x7000



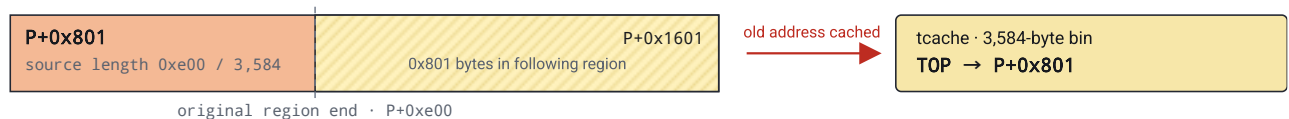
THE SELECTED REGION



THE 4,001-DIGIT GROWTH



WHAT JEMALLOC COPIES AND CACHES



Observed in the profiled GitLab 18.11.3 / jemalloc build; passing an interior pointer to `realloc` is allocator-specific behavior.

The 4,001-digit growth sends the forged interior pointer through `realloc` and leaves `P+0x801` at the top of the 3,584-byte tcache bin.

The Overlap: A Ruby Array Reclaims `P+0x801`

Writer 2 needed a normal 3,584-byte-class allocation whose initialization copied attacker-selected data. Closing an inner JSON array with 446 values provided exactly that allocation. Oj called `rb_ary_new_from_values(446, values)` . Each

bytes and landed in the same 3,584-byte class.

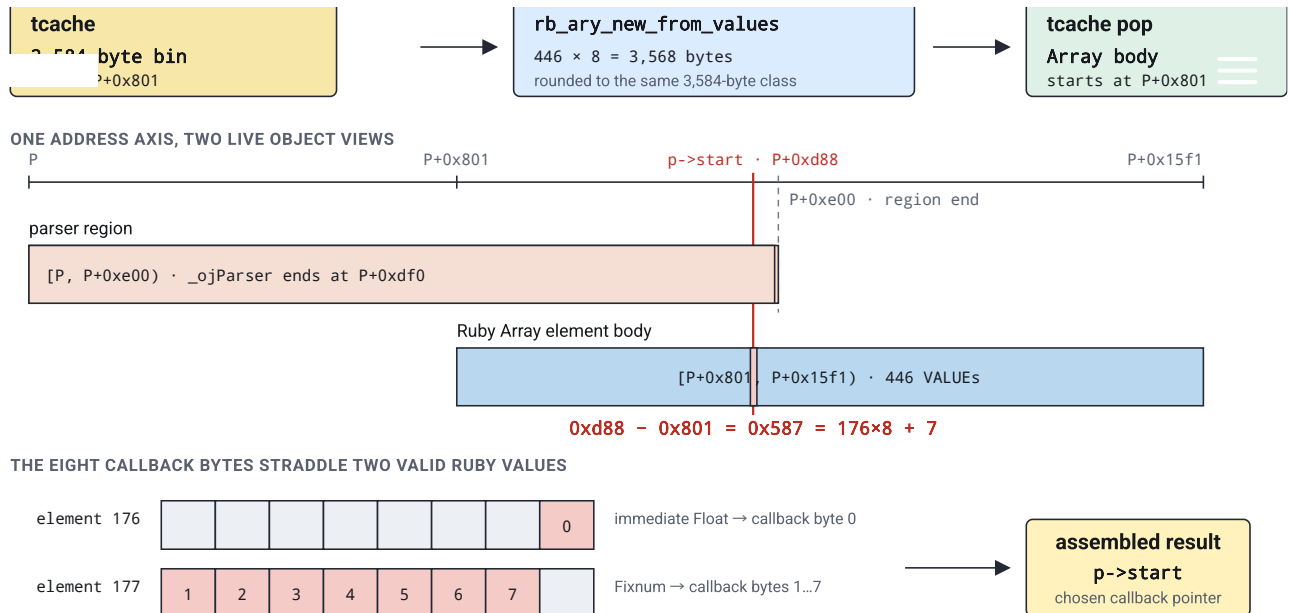
Closing the large integer also created Ruby String storage and a BigDecimal before Oj opened the inner Array. In this build, the String buffers used the 4,096-byte class, BigDecimal's roughly 1.8 KiB Real used the 2,048-byte class, and the moved numeric buffer occupied the 6,144-byte class. Oj's preallocated 4,096-entry VALUE stack still held the roughly 2,580 live entries, the collection stack had already grown, and all 446 inner elements were immediate values. None of these operations consumed the 3,584-byte tcache entry. Because parsing and Array construction remained under the GVL on one native thread, `P+0x801` was still at the top of that bin when CRuby allocated the Array body.

The Ruby Array object came from the GC heap, while its external element body came from jemalloc. That body began at the reclaimed `P+0x801` and occupied `[P+0x801, P+0x15f1)`. The write stayed inside the 3,584-byte range already proven mapped by the successful realloc copy.

The Array copy was Writer 2, not the nesting overflow. It supplied the callback bytes. Its destination was fixed, its useful length was 3,568 bytes, and every eight-byte word had to be a valid Ruby VALUE. At body offset `0x587`, those constraints were still enough to control `p→start`: element 176 byte 7 overlapped callback byte 0, and element 177 bytes 0 through 6 overlapped callback bytes 1 through 7.

The chain had therefore amplified a forward sweep of one-byte selectors into an allocator-assisted overlapping-object write with a chosen callback pointer under CRuby encoding constraints.

Встретимся на Black Hat USA →



A 446-element Ruby Array reclaims P+0x801 and places p→start across two adjacent VALUEs.

The Controlled Field: Two Ruby VALUE s Assemble p→start

Two adjacent Ruby VALUE s were enough to assemble the callback pointer. Immediate Floats and Fixnums provided useful bit patterns without requiring separately allocated Ruby objects.

On little-endian x86_64, element 176 byte 7 became p→start byte 0, while element 177 bytes 0 through 6 became callback bytes 1 through 7. For the final libruby target used after ASLR recovery, the immediate Float literal `-1e45` had the representation `0x4a335dbf821ae4f6`. Its most-significant byte supplied the address's first byte, `0x4a`.

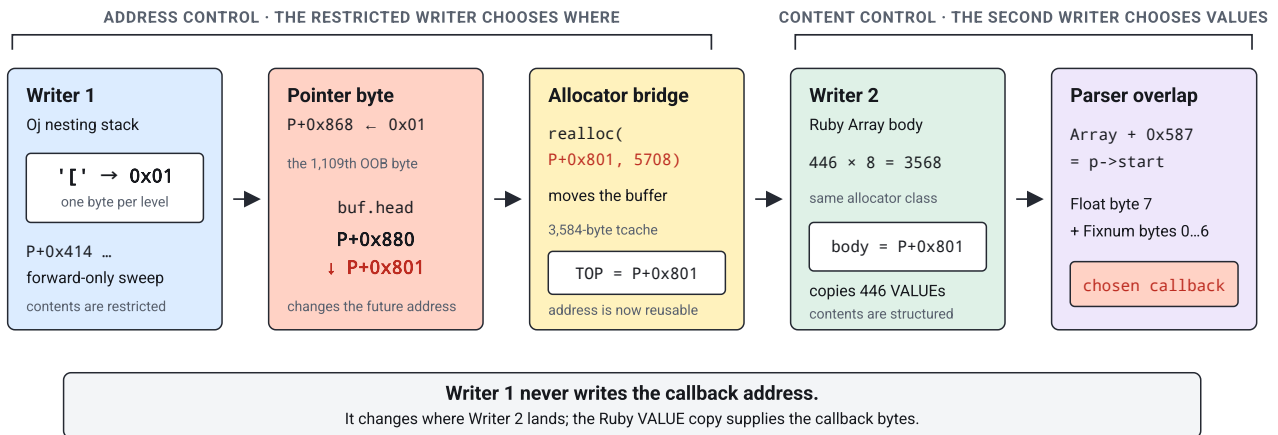
Ruby encodes a small integer as the tagged Fixnum $\text{VALUE} = (\text{integer} \ll 1) | 1$. The target's second address byte was odd, satisfying the tag bit, and the remaining seven bytes were encoded into element 177 with:

```
integer = (((target >> 8) - 1) >> 1)
```

The other Array entries were immediate integer fillers. When

`rb_ary_new_from_values` copied the 446 words into the reclaimed body, elements 176 and 177 assembled the chosen native address at `p→start`.

before the corrupted `p→result` callback could run. The exception also skipped SymbolMap SAJ parse. On the next invocation, `parser_reset` cleared per-parse state but did not restore `p→start`, so the overwritten callback survived.



restricted byte sweep → forged pointer → reusable allocation → structured overwrite → `p->start` control

Result: selected function-pointer control, not a general arbitrary-address / arbitrary-byte primitive.

The complete write-amplification chain shows how the restricted selector sweep determines where the later Ruby VALUE copy lands.

The write chain could now place an address satisfying CRuby's VALUE -encoding constraints in `p→start`. ASLR still hid which library address to use.

II. Read: Turning a Heap Pointer into an ASLR Base

With control of `p→start` established, the remaining problem was locating `libc` and `libruby` under ASLR. An oversized object key made Oj return a fixed 29-byte slice of one `Key` entry, and GitLab carried that slice into the rendered diff. Eight of those bytes were the pointer to Oj's duplicated key allocation. The primitive could not choose or dereference another address, but it provided a heap landmark for the callback overwrite developed in Part I.

One Key, Two Layouts

Oj's usual parser stored each object key in a 32-byte `union _key`. The [source](#) used two anonymous struct views over that storage:

```

    struct {
        int16_t len;
        char    buf[30];
    };
    struct {
        int16_t xlen; // same storage as len
        char    *key;
    };
} *Key;

```

For clarity, these are the inline view and the external view. `len` and `xlen` occupied the same two bytes at union offset 0. The inline buffer required only byte alignment and began at offset 2. The external pointer required eight-byte alignment, so the x86_64 ABI inserted six bytes of padding and placed `key` at offset 8. Read through the inline view, those pointer bytes appeared at `buf[6]` through `buf[13]` .

The stored `len` selected the view later code used. A value below 30 selected `buf` ; a longer value selected `key` . For a long input, Oj duplicated the key on the heap and stored the resulting `char *` through the external view.

The parser received the real key length as a `size_t` , then assigned it to the signed 16-bit `len` field without enforcing a compatible upper bound. The allocation branch still used the original `size_t` length, so Oj stored the duplicated pointer. Later, the truncated `len` made the return path read the inline buffer instead. The pointer form had been written, but the inline form was read.

The Pointer Hidden at Byte Six

The useful input length was `0x10000 + 29` , or 65,565 bytes. On the tested GCC/Clang x86_64 ABI, narrowing that value to `int16_t` left 29. Later code saw `29 < 30` and returned 29 bytes through the inline `buf` . That read began at union offset 2 and covered offsets 2 through 30.

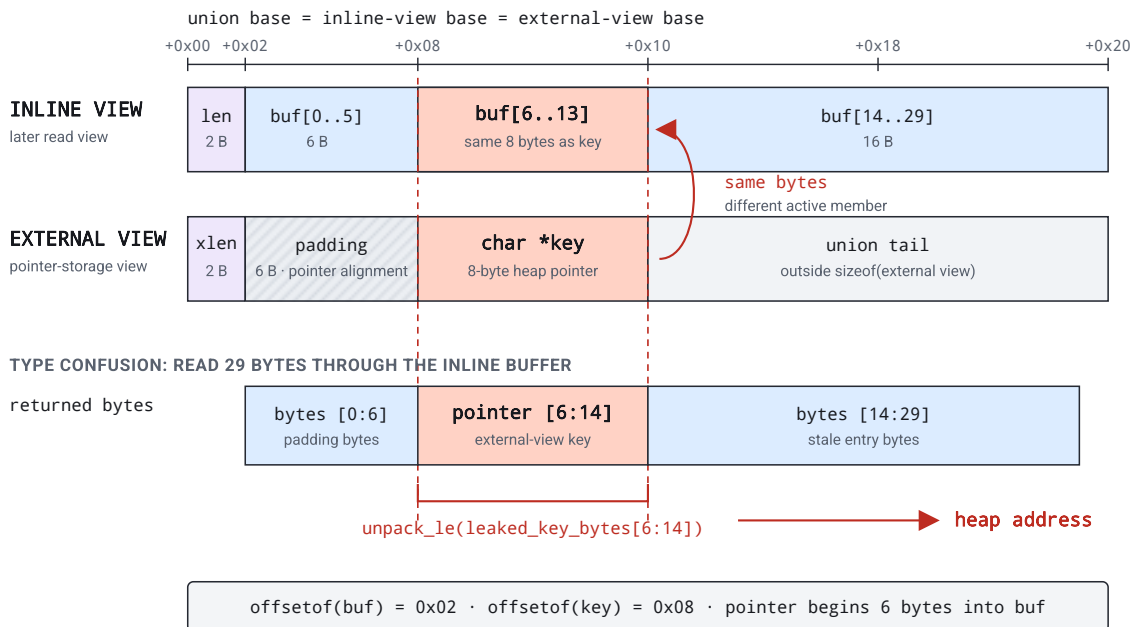
The live external `key` pointer still occupied union offsets 8 through 15. Relative to a read beginning at offset 2, the pointer therefore appeared six bytes into the

```
pointer = unpack_le(leaked_key_bytes[6:14])
```

Returned bytes 0 through 5 came from the external view's alignment padding. Bytes 6 through 13 were the live `key` pointer. The final 15 returned bytes came from union storage not written by the external view and could contain stale key-entry contents. This was a fixed 29-byte disclosure, not an arbitrary-address read: it exposed bytes already present in that `Key` entry and never dereferenced an attacker-selected pointer. These offsets came from the tested x86_64 ABI, so the compiler layout was part of the primitive's boundary.

The same truncated length controlled cleanup. Oj freed `kp→key` only when `kp->len ≥ 30`; with the stored length equal to 29, that free was skipped. The disclosed address therefore still referred to the duplicated 65,565-byte key allocation.

SAME 32 BYTES, TWO STRUCT VIEWS



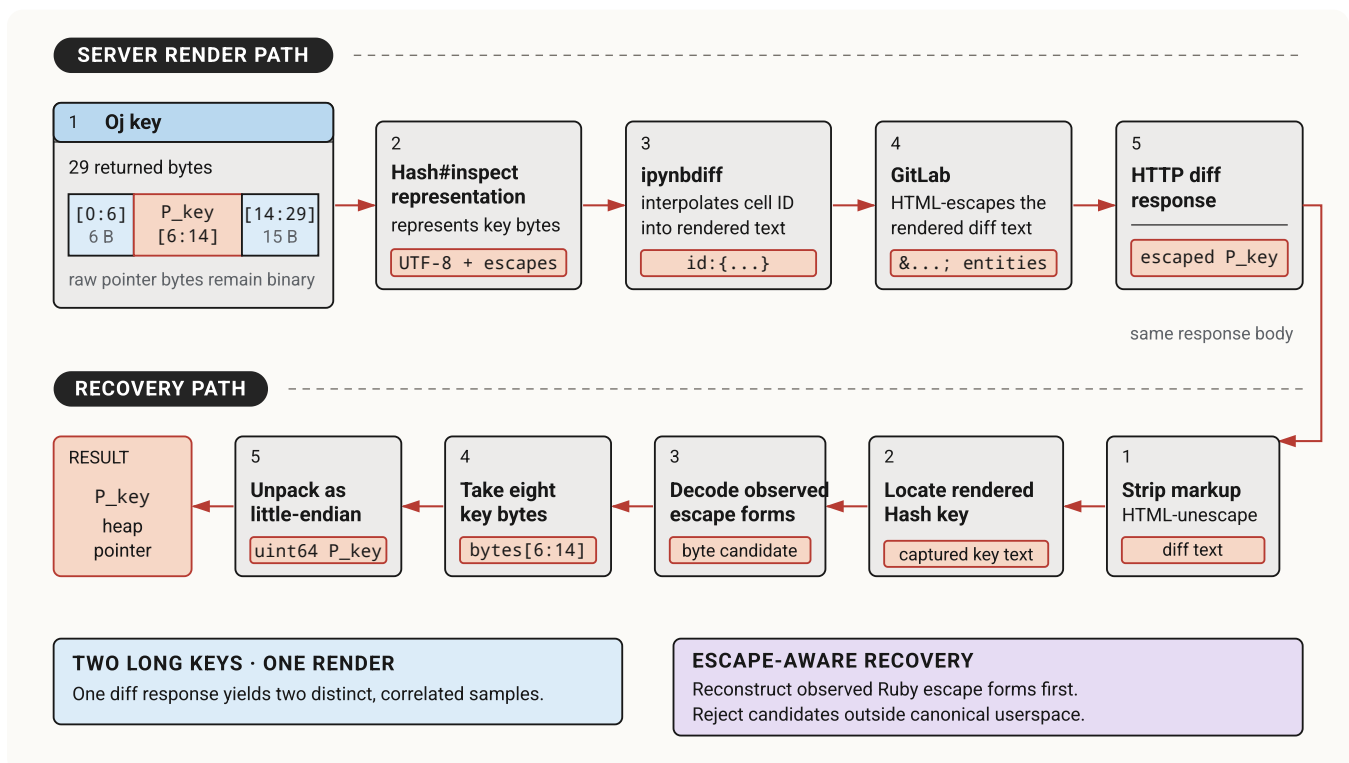
Oj's anonymous inline and external views share one 32-byte union; pointer alignment places key at +0x08, overlapping buf[6..13].

A Pointer Wrapped in Ruby, Then HTML

The notebook transformer included each cell ID in its human-readable output:

The notebook encoded `cell['id']` as a JSON object with two 65,565-byte keys. Oj materialized that object as a Ruby `Hash`; interpolation used its `Hash#inspect` representation, and GitLab then HTML-escaped that text inside the rendered diff.

Recovery removed the diff markup, decoded the HTML entities and the escape forms observed in the response, extracted bytes 6 through 13, and retained only canonical userspace pointer-shaped results.



The rendered diff contains a `Hash#inspect` representation inside HTML; recovery reverses the observed encodings before validating the pointer candidate.

1 file +4 -1
↕ ✖ ⚙

leak.ipynb
+1 -1

```

3  - %% Cell type:markdown id:baseline tags:
3  + %% Cell type:markdown id:
    + {"nbformat": 4, "nbformat_minor": 2, "cells": [
    +   {
    +     "cell_type": "code",
    +     "execution_count": 1,
    +     "metadata": {
    +       "id": "sample-a",
    +       "name": "sample-a",
    +       "tags": [
    +         "sample-a"
    +       ]
    +     },
    +     "source": [
    +       "sample-a"
    +     ]
    +   },
    +   {
    +     "cell_type": "code",
    +     "execution_count": 2,
    +     "metadata": {
    +       "id": "sample-b",
    +       "name": "sample-b",
    +       "tags": [
    +         "sample-b"
    +       ]
    +     },
    +     "source": [
    +       "sample-b"
    +     ]
    +   }
    + ], "metadata": {
    +   "id": "sample-a",
    +   "name": "sample-a",
    +   "tags": [
    +     "sample-a"
    +   ]
    + }, "nbformat": 4, "nbformat_minor": 2}
    +
8  11  Rendered notebook diff

```

Please [register](#) or [sign in](#) to reply

GitLab's rendered notebook diff after Oj materialized the JSON cell ID as a Hash. Each reconstructed key places the live heap pointer at bytes 6–13.

from one render. Because the truncated cleanup path retained each duplicated key allocation, two keys per render increased coverage while keeping the number of heap-perturbing requests low. Repeating the request sampled the Puma worker population.

A Heap Landmark Inside a Forked Worker Tree

The leaked address pointed to the copied key. It was a heap landmark, while the final gadgets lived in `libc` and `libruby`. The useful relationship was the displacement between that landmark and the library mappings:

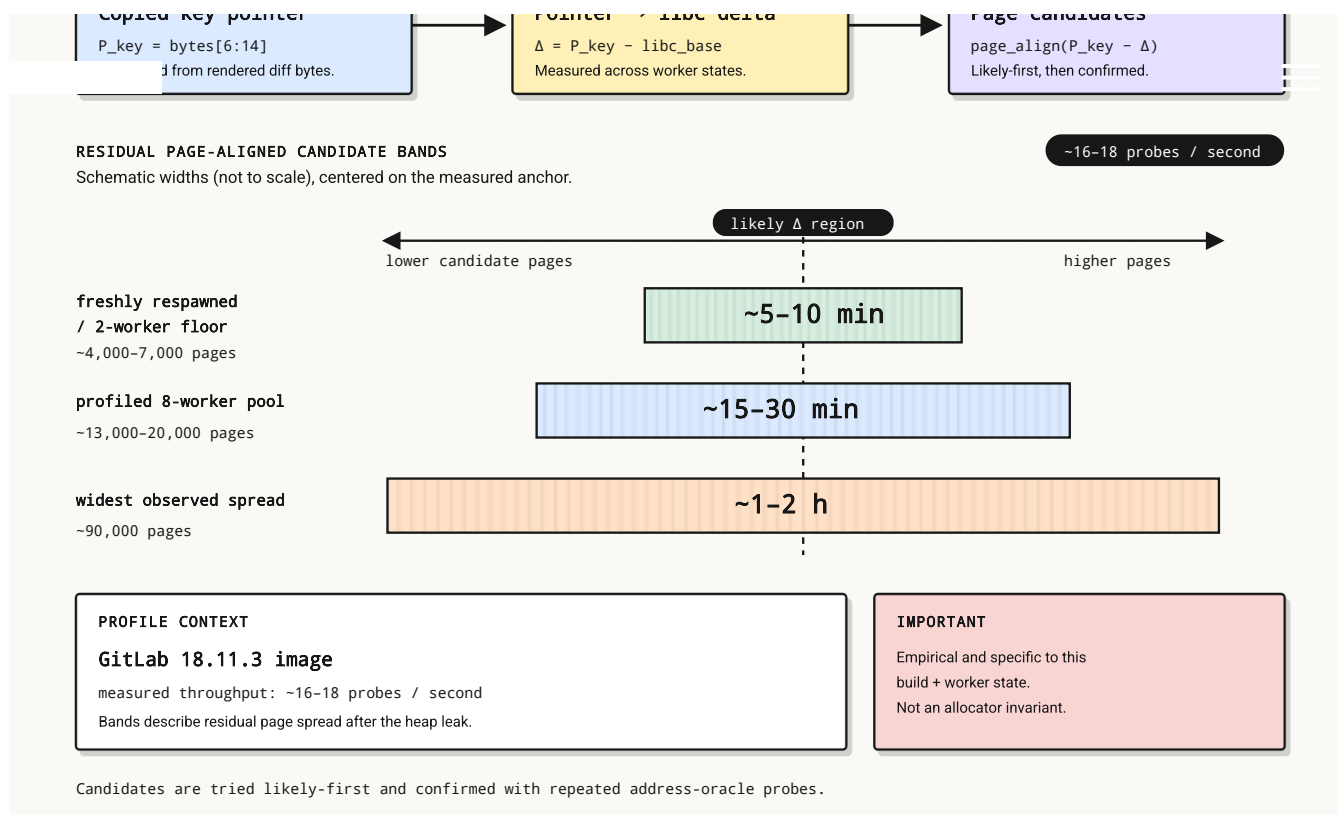
```
delta = leaked_key_pointer - libc_base
```

In the profiled GitLab 18.11.3 Omnibus container, Puma ran in preloaded cluster mode. The master established the `libc`, `libruby`, `jemalloc`, and `Oj` mappings before forking its workers, so workers in that process tree inherited identical library addresses. Their heaps diverged as they served requests. A replacement worker forked by the same master inherited the same mappings; separate Puma masters and master restarts had independent layouts.

In the profiled GitLab image, the absolute heap-to-`libc` displacement was roughly 2–3 GiB. That magnitude did not determine the search cost. The uncertainty in `delta`, created by different worker heap histories, determined how many page candidates remained.

Puma Sets the Search Band

Repeated render requests sampled fresh and mature workers. Across those measurements, the highest valid pointer sample tracked the freshest worker and served as `maxP`, the empirical anchor for the candidate band. This relationship belonged to the profiled target and worker lifecycle; it was not a `jemalloc` invariant.



Empirical ASLR search bands on the profiled GitLab 18.11.3 image. The 5–10 minute result was measured in the two-worker fresh configuration; wider bands are projected from the measured probe rate.

The leak reduced ASLR to a finite set of page-aligned libc candidates. The callback overwrite from Part I supplied the yes-or-no test. Each probe wrote `candidate_base + 0x8470e`, the profiled libc offset of an `eb fe ( jmp $ )` instruction, to `p→start` and invoked it on the next parse in the same request. Page alignment made the target's first byte `0x0e` and its second byte `0x47` for every candidate. The immediate Float `540000000.0` supplied byte 0, while the odd second byte allowed the Fixnum to encode bytes 1 through 7. A correct candidate entered the self-loop and held the diff request open. Every timeout candidate was probed twice more before acceptance.

The search began around the most likely displacement and expanded across the measured band concurrently. Two-key sampling, likely-first ordering, and the confirmation pass reduced variance. In the profiled two-worker fresh configuration for GitLab 18.11.3, the search usually completed in about 5–10 minutes. Extrapolating the measured probe rate across the widest mature-worker band gave roughly 1–2 hours.

short register-only sequence ending in `ret`. When `p→start` landed on one, it turned to the parser and the diff request completed with HTTP 200. The offsets between these 200-producing sites were precomputed for the exact binaries. After the first 200, a few probes at the expected companion offsets identified the gadget and recovered the mapping base. Unlike the original `jmp $` oracle, which had to keep testing candidate bases until the single hanging address was reached, this method could identify the mapping from the first 200 signature and a few follow-up probes.

Once recovered, the base remained valid for that forked worker tree until its Puma master restarted. Part I supplied control of `p→start`, and Part II supplied the address to place there.

Chaining the Primitives into RCE

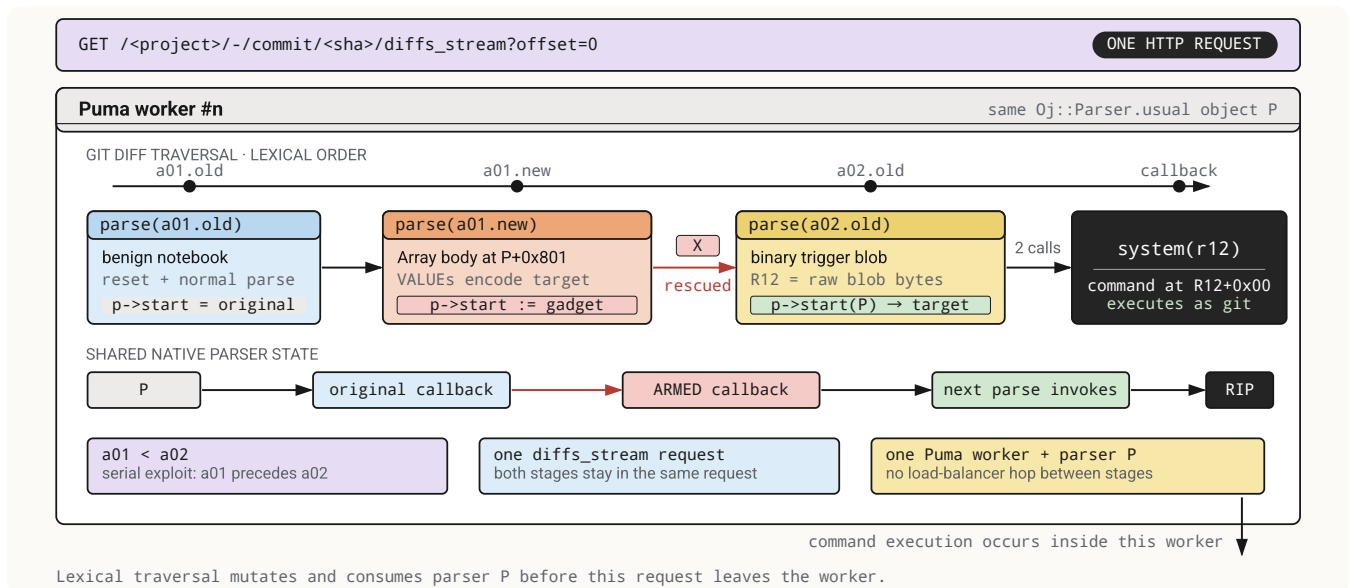
In the profiled image, `libruby.so` began `0x661000` above `libc`. Recovering the `libc` base therefore also located the final `libruby` callback sequence, while `system()` came from the same `libc` mapping. The self-loop probe could now be replaced with that target; the corrupting parse and command-bearing trigger still had to reach the same process-local `Oj` parser in order.

Keeping Both Parses in One Puma Worker

The trigger had one process boundary: the next parse had to use the same native parser object. `Oj::Parser.usual` returned a process-global parser singleton; threads in one Puma worker shared that object. A single `diffs_stream` request could transform several notebook files without crossing into another worker.

Two lexically ordered files carried the stages. For `a01`, the old side was benign and the new side contained the corruptor followed by `x`. For `a02`, the old side was a binary trigger blob and the new side was a different benign notebook. The old `a02` blob was stored byte-for-byte and passed to `Oj` as a binary-safe `Ruby String`. It could therefore contain a NUL-terminated command and little-endian native pointers in the same blob; the `.ipynb` path still selected the notebook renderer.

notebook retained the old-then-new transform order described earlier. The made `a01.new` raise `InvalidNotebookError` after arming the callback. GitLab rescued that exception in the per-file notebook renderer and continued streaming the remaining diff files. With exploit requests delivered serially in the profiled environment, traversal then reached `a02`; its old side was transformed first, making `a02.old` the next `Oj::Parser.usual.parse` in that request and Puma worker.



In the serial exploit sequence, one `diffs_stream` request arms `p->start` in `a01` and invokes it from `a02` in the same Puma worker.

The Next Parse Calls the Overwritten Pointer

Oj invoked the callback at the beginning of each parse:

```
parser_reset(p);
p->start(p);
parse(p, ptr);
```

Encoding `0x0000414141414141` into Array elements 176 and 177 placed the same value in the processor's instruction pointer on the next notebook parse. GitLab independently reproduced the overwrite and reliable control transfer to the chosen callback address.

```
Ruby thread count for this reactor: 20
```

```

----- Register context -----
RIP: 0x0000414141414141 RBP: 0x0000707d5e57c800 RSP: 0x0000707d5e57c7d0
RAX: 0x0000707dbdcef0c0 RBX: 0x0000707d4e114200 RCX: 0x0000707dbdcce780
RDX: 0x0000707d4c514768 RDI: 0x0000707d4c514768 RSI: 0x0000707dbdcf8080
R8: 0x0000707d4cdef4a0 R9: 0x0000000000000001 R10: 0x0000000000000000
R11: 0x0000707d4ccf1128 R12: 0x0000707d4c514768 R13: 0x0000707dc05b51d0
R14: 0x0000707dbdc46660 R15: 0x0000707d7f30ef40 EFL: 0x0000000000010246
root@gl-fresh:/# sed -n '1456051,1456209p' /var/log/gitlab/puma/puma_stderr.log
/opt/gitlab/embedded/service/gitlab-rails/gems/ipynbdiff/lib/ipynb_diff/transformer.rb:27: [BUG] Segmentation fault at 0x0000414141414141
ruby 3.3.10 (2025-10-23 revision 343ea05002) [x86_64-linux]

-- Control frame information -----
c:0069 p:---- s:0441 e:000440 CFUNC :parse
c:0068 p:0008 s:0436 e:000435 METHOD /opt/gitlab/embedded/service/gitlab-rails/gems/ipynbdiff/lib/ipynb_diff/transformer.rb:27
c:0067 p:0014 s:0430 e:000429 METHOD /opt/gitlab/embedded/service/gitlab-rails/gems/ipynbdiff/lib/ipynb_diff/transformer.rb:39
c:0066 p:0027 s:0422 e:000420 METHOD /opt/gitlab/embedded/service/gitlab-rails/gems/ipynbdiff/lib/ipynb_diff.rb:12
c:0065 p:0029 s:0409 e:000408 BLOCK /opt/gitlab/embedded/service/gitlab-rails/lib/gitlab/diff/rendered/notebook/diff_file.rb:88
c:0064 p:0027 s:0406 e:000405 BLOCK /opt/gitlab/embedded/lib/ruby/gems/3.3.0/gems/timeout-0.4.3/lib/timeout.rb:185
c:0063 p:0012 s:0401 e:000400 METHOD /opt/gitlab/embedded/lib/ruby/gems/3.3.0/gems/timeout-0.4.3/lib/timeout.rb:38
c:0062 p:0124 s:0394 e:0010c0 METHOD /opt/gitlab/embedded/lib/ruby/gems/3.3.0/gems/timeout-0.4.3/lib/timeout.rb:194

```

Puma crash log showing controlled RIP at 0x0000414141414141.

From p→start to system()

In the Oj binary shipped with the profiled GitLab build, the compiler kept the raw pointer returned by `rb_string_value_ptr` in R12 until the callback:

```

call  rb_string_value_ptr
mov   r12, rax
...
call  qword ptr [rbx+0xd88]

```

At callback entry, R12 pointed to the raw bytes of `a02.old`; RBX and RDI held parser P. That build-specific register state made the trigger blob useful as both data and a small table of function pointers.

The verified callback target was `G1 = libc_base + 0x661000 + 0x269b4a`, or `libruby + 0x269b4a`:

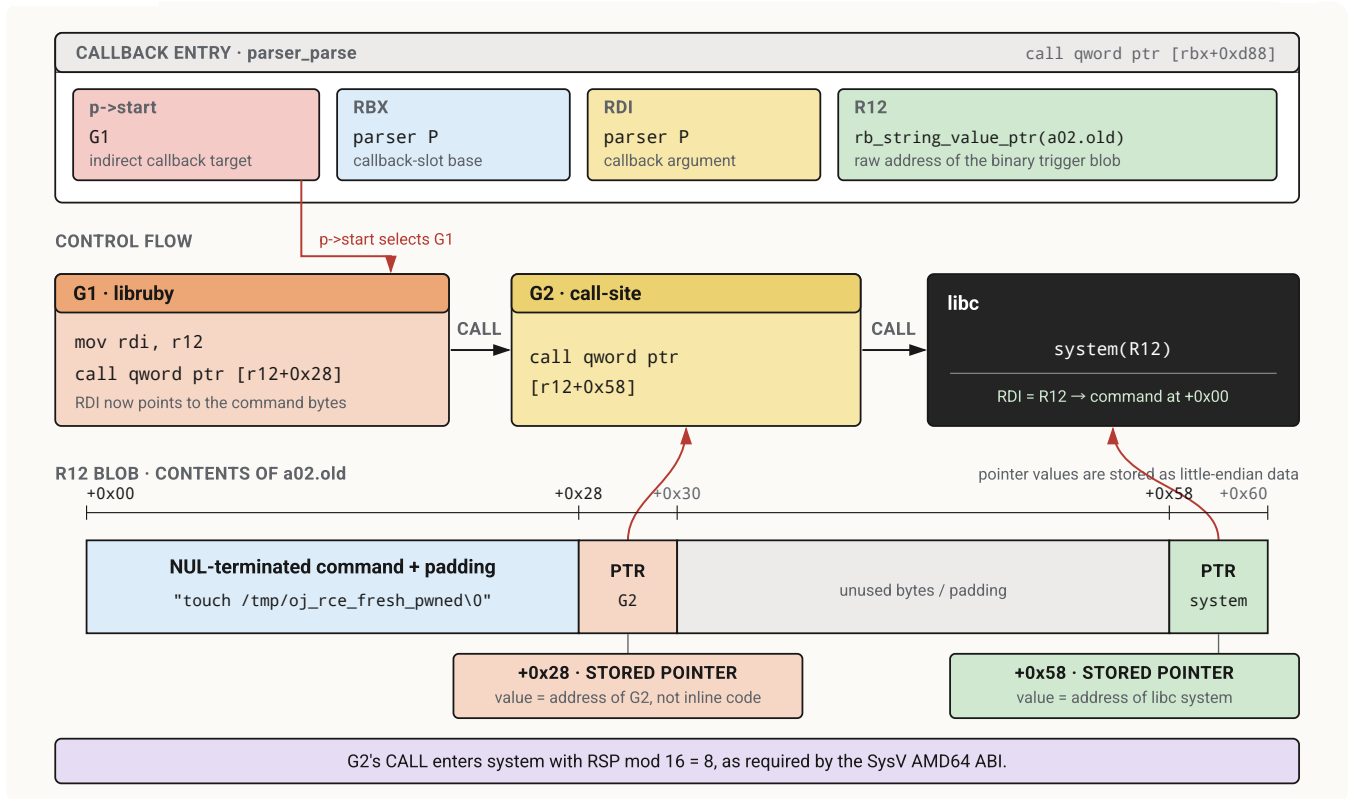
```

mov   rdi, r12
call  qword ptr [r12+0x28]

```

The blob stored the NUL-terminated command at `R12+0x00`, `G2 = libruby + 0x22d565` at `R12+0x28`, and `system = libc + 0x58750` at `R12+0x58`. The

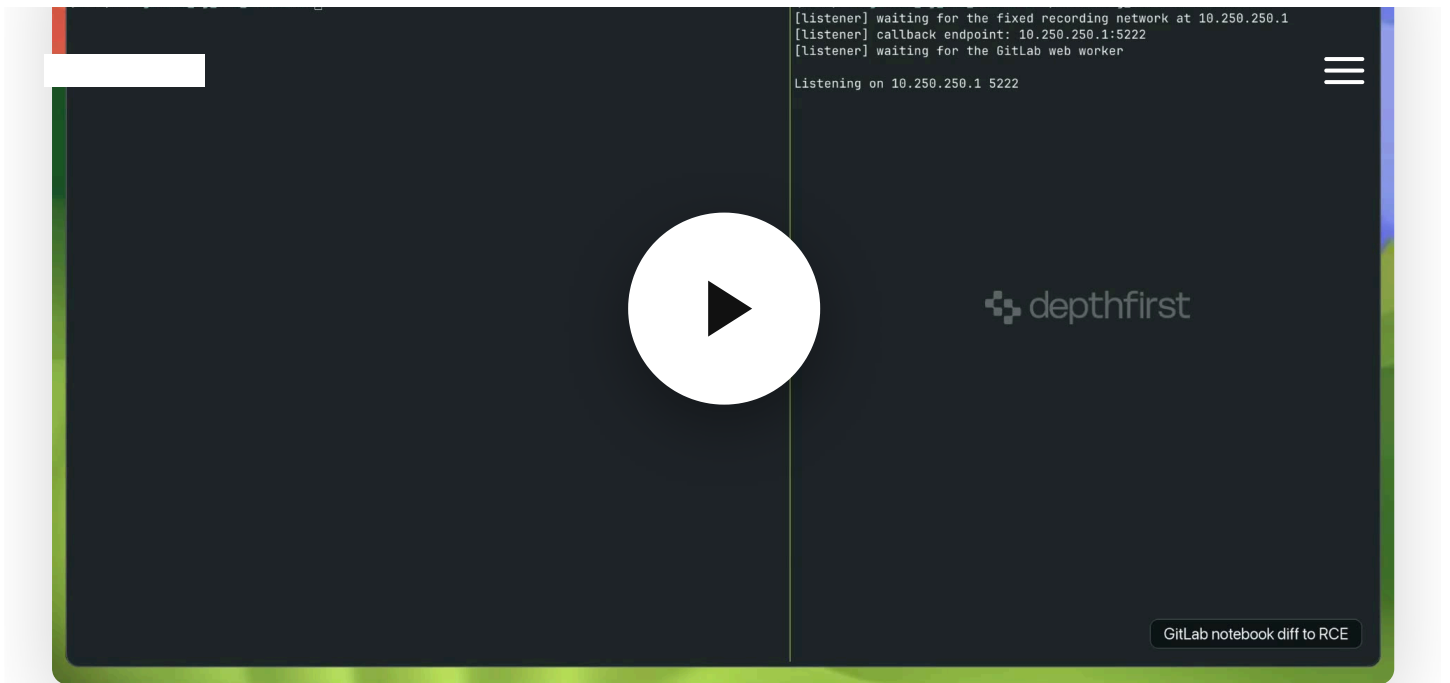
39 command bytes in this compact layout.



R12 carries the command and two stored function pointers through build-specific libruby call sites into system().

The first gadget moved the command buffer into RDI and called the pointer at offset 0x28. G1 was itself entered by a call, so calling system() directly from G1 would use the wrong 16-byte stack phase. Entering G2 added another eight-byte return address; G2's call then entered system(R12) with RSP mod 16 = 8, as required by the SysV AMD64 ABI.

The register assignments, mapping delta, and gadget offsets came from the exact libruby and compiler build used by the profiled GitLab image.



End-to-end GitLab notebook-diff remote code execution demonstration.

The marker command ran as `git`, the account serving the Puma application. With the library base recovered in Part II, the corruption, callback overwrite, and trigger completed in seconds. CRuby `VALUE` encoding, jemalloc behavior, gadget offsets, and library deltas were build-specific; the two vulnerable Oj parser bugs spanned the GitLab releases described below.

Impact

A normal authenticated user who could push to a project and view its commit diff could reach this path from their own project. Exploitation required no administrator privileges, CI or runner access, victim interaction, or access to another user's project.

Successful exploitation executed commands as `git`, the account running GitLab's Puma workers. Its effective reach depends on deployment isolation, but it may include repository data, Rails secrets, service credentials, and internal services available to the application. This can lead to source-code disclosure, credential theft, repository modification, persistence, or lateral access.

Affected Versions

Edition and is not restricted by subscription tier. A stable self-managed release is affected when it contains the `Oj::Parser.usual.parse` notebook path and bundles Oj 3.13.0 through 3.17.1.

Component	Affected versions	First fixed release
GitLab CE/EE	15.2.0 through 18.10.7	18.10.8
GitLab CE/EE	18.11.0 through 18.11.4	18.11.5
GitLab CE/EE	19.0.0 through 19.0.1	19.0.2
Oj published gems	3.13.0 through 3.17.1	3.17.3

GitLab 15.1 and earlier used `JSON.parse` in this notebook path and are outside this chain. Releases from 15.2 through 18.9 did not receive dedicated backports because they were already outside GitLab's [security-maintained patch trains](#); those installations must move to a supported fixed release.

The same application ranges apply to Linux packages, Docker images, source installations, Geo, and cloud-native deployments. For Helm, Operator, and custom-image installations, the GitLab version in the Webservice image running Puma is authoritative. Under the [default chart mappings](#), the first fixed charts are 9.10.8, 9.11.6, and 10.0.2 for the three fixed branches above; image overrides take precedence.

GitLab stated in the [June 10 patch release](#) that GitLab.com was already patched and Dedicated customers did not need to act. For release candidates, downstream packages, and forks, check the two source conditions above rather than relying only on a version label.

Oj 3.17.3 was the first published gem containing both parser fixes. Self-managed operators should upgrade to a current supported GitLab patch release.

The Broader Oj Review

In addition to the two parser bugs used in this chain, the broader Oj review resulted in nine published advisories across the parser, loader, dumper, and document APIs.

- [CVE-2026-54502](#): stack buffer overflow in `Oj.dump` with a large indent.
- [CVE-2026-54896](#): heap buffer overflow during exception serialization with a large indent.
- [CVE-2026-54897](#): use-after-free in `Oj::Doc` iterators during reentrant close.
- [CVE-2026-54898](#): use-after-free in an SAJ callback after input mutation.
- [CVE-2026-54899](#): use-after-free when toggling the parser's symbol-key cache.
- [CVE-2026-54900](#): negative-size `memcpy` in `create_id` attribute handling.
- [CVE-2026-54901](#): use-after-free from missing GC marking for `array_class` and `hash_class`.
- [CVE-2026-54902](#): use-after-free in the SAJ long-key callback.
- [CVE-2026-54903](#): integer overflow while loading a JSON string larger than 2 GiB.

Timeline

8/8/2021: Oj's new parser was merged with the unchecked nesting write and unsafe 16-bit key-length handling. Oj 3.13.0 was the first release containing that parser.

7/7/2022: GitLab changed notebook validation from `JSON.parse` to `Oj::Parser.usual.parse`; the change shipped in GitLab 15.2.0.

5/21/2026: The issues were reported to Oj.

5/27/2026: Oj merged the nesting-depth and overlong-key fixes after the vulnerable code had remained in the parser for 1,753 days.

6/4/2026: Oj 3.17.3 became the first published release containing the fixes.

6/5/2026: The remotely reachable notebook-diff chain was privately reported to GitLab.

6: GitLab released 19.0.2, 18.11.5, and 18.10.8 with Oj 3.17.3.

7/17/2026: GitLab resolved the report.

Secure on depthfirst

[Request demo](#)



Product

[Platform](#)

[Code](#)

[Supply chain](#)

[Secrets](#)

[Agentic Pentesting](#)

Customers

[Customer stories](#)

[Moveworks](#)

[Persona](#)

[AngelList](#)

Resources

[Events](#)

[Blog](#)

[Research](#)

[Book a demo](#)

[Trust center](#)

Company

[About](#)

[Careers](#)

